Lab 5: Static Code Analysis

To-Do List & Task Manager (`todo_manager.py`)

Name	Shivansh Sharma
SRN	PES2UG24CS474
University	PES University, Bengaluru
Assigned repo	L5_37_todo_manager
Program	todo_manager.py
Tools	Pylint, Bandit, Flake8

1. Results summary

Tool	Before	After
Pylint	5.09 / 10	**10.00 / 10** (target: above 9.5)
Bandit	4 issues: 1 High (B605), 1 Medium (B301), 2 Low (B403, B311)	**No issues identified**
Flake8	4 violations: E225, E303, E261, E231	**No output (clean)**
Program output	Reference run of the original code	Identical output after all fixes

2. Issues found and fixes

Issue	Type	Line(s)	Description	Fix
Shell command injection `os.system` (B605)	Security, High	57	The `address` argument is concatenated into a shell command, so input such as `a@b.com; rm -rf /` would run arbitrary commands.	Removed `os.system` and `import os`; build the message with `email.message.EmailMessage` (no shell).
Insecure deserialization `pickle` (B301, B403)	Security, Medium	3, 47	`pickle.load` can execute arbitrary code from a tampered `tasks.pkl`.	Replaced `pickle` with `json.dump` / `json.load`; removed `import pickle`.
Mutable default argument (W0102)	Bug	11	`labels=[]` is created once and shared by every call that omits it.	Default changed to `None`; a new list is created inside the function.
Weak PRNG `random.choice` (B311)	Security, Low	70	Mersenne Twister is predictable and not suitable for security use.	Replaced with `secrets.choice`.
Inconsistent return (R1710)	Quality	33	`findTask` returns an id on a match but falls off the end otherwise.	Added an explicit `return None`.
No file encoding (W1514)	Portability	51	`open()` without `encoding` depends on the OS default.	Added `encoding="utf-8"` to every `open()`.
PEP 8 spacing (E225, E303, E261, E231)	Style	12, 40, 69, 74	Missing space around =, 3 blank lines, one space before an inline comment, missing space after a comma.	Fixed whitespace and blank lines.
camelCase names (C0103)	Style	11-55	Functions and variables such as `addTask`, `taskId`, `fileName` break snake_case.	Renamed to `add_task`, `task_id`, `file_name`, etc.
Missing docstrings (C0114, C0116)	Docs	1, 11-60	Neither the module nor any function had a docstring.	Added a module docstring and one for every function.

Notes: `remind_by_email` now builds the email message but does not send it (it is not called by the demo). `save_tasks` now writes JSON, so the program no longer creates `tasks.pkl`. A `labels` key is now stored on each task.

3. Key code changes

Shell injection (B605)

```
BEFORE
def remindByEmail(address):
    os.system("mail -s 'Pending tasks' " + address + " < tasks.json")

AFTER
def remind_by_email(address):
    with open("tasks.json", "r", encoding="utf-8") as file:
        content = file.read()
    msg = EmailMessage()
    msg.set_content(content)
```

```
    msg["Subject"] = "Pending tasks"
    msg["To"] = address
    return msg
```

Pickle and mutable default

```
BEFORE
def loadTasks(fileName="tasks.pkl"):
    with open(fileName, "rb") as f:
        return pickle.load(f)
def addTask(title, priority="medium", labels=[]):

AFTER
def load_tasks(file_name="tasks.json"):
    with open(file_name, "r", encoding="utf-8") as file:
        data = json.load(file)
        return {int(k): v for k, v in data.items()}
def add_task(title, priority="medium", labels=None):
    if labels is None:
        labels = []
```

4. Tool output: before

Pylint (before)

```
************* Module todo_manager
todo_manager.py:1:0: C0114: Missing module docstring (missing-module-docstring)
todo_manager.py:11:0: C0116: Missing function or method docstring (missing-function-docstring)
todo_manager.py:11:0: C0103: Function name "addTask" doesn't conform to snake_case naming style (invalid-
    name)
todo_manager.py:11:0: W0102: Dangerous default value [] as argument (dangerous-default-value)
todo_manager.py:12:4: C0103: Variable name "taskId" doesn't conform to snake_case naming style (invalid-
    name)
todo_manager.py:19:0: C0116: Missing function or method docstring (missing-function-docstring)
todo_manager.py:19:0: C0103: Function name "completeTask" doesn't conform to snake_case naming style
    (invalid-name)
todo_manager.py:19:17: C0103: Argument name "taskId" doesn't conform to snake_case naming style (invalid-
    name)
todo_manager.py:27:0: C0116: Missing function or method docstring (missing-function-docstring)
todo_manager.py:27:0: C0103: Function name "pendingTasks" doesn't conform to snake_case naming style
    (invalid-name)
todo_manager.py:33:0: C0116: Missing function or method docstring (missing-function-docstring)
todo_manager.py:33:0: C0103: Function name "findTask" doesn't conform to snake_case naming style (invalid-
    name)
todo_manager.py:34:8: C0103: Variable name "taskId" doesn't conform to snake_case naming style (invalid-
    name)
todo_manager.py:33:0: R1710: Either all return statements in a function should return an expression, or none
     of them should. (inconsistent-return-statements)
todo_manager.py:40:0: C0116: Missing function or method docstring (missing-function-docstring)
todo_manager.py:40:0: C0103: Function name "saveTasks" doesn't conform to snake_case naming style (invalid-
    name)
todo_manager.py:40:14: C0103: Argument name "fileName" doesn't conform to snake_case naming style (invalid-
    name)
todo_manager.py:45:0: C0116: Missing function or method docstring (missing-function-docstring)
todo_manager.py:45:0: C0103: Function name "loadTasks" doesn't conform to snake_case naming style (invalid-
    name)
todo_manager.py:45:14: C0103: Argument name "fileName" doesn't conform to snake_case naming style (invalid-
    name)
todo_manager.py:50:0: C0116: Missing function or method docstring (missing-function-docstring)
todo_manager.py:50:0: C0103: Function name "exportJson" doesn't conform to snake_case naming style (invalid-
    name)
todo_manager.py:50:15: C0103: Argument name "fileName" doesn't conform to snake_case naming style (invalid-
    name)
todo_manager.py:51:9: W1514: Using open without explicitly specifying an encoding (unspecified-encoding)
todo_manager.py:55:0: C0116: Missing function or method docstring (missing-function-docstring)
todo_manager.py:55:0: C0103: Function name "remindByEmail" doesn't conform to snake_case naming style
    (invalid-name)
todo_manager.py:60:0: C0116: Missing function or method docstring (missing-function-docstring)

-----------------------------------
Your code has been rated at 5.09/10
```

Bandit (before)

```
Run started:2026-10-14 06:24:32.408152+00:00

Test results:
>> Issue: [B403:blacklist] Consider possible security implications associated with pickle module.
   Severity: Low   Confidence: High
   CWE: CWE-502 (https://cwe.mitre.org/data/definitions/502.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_imports.html#b403-import-pickle
   Location: .\todo_manager.py:3:0
2       import os
3       import pickle
4       import random

--------------------------------------------------
>> Issue: [B301:blacklist] Pickle and modules that wrap it can be unsafe when used to deserialize untrusted
    data, possible security issue.
   Severity: Medium   Confidence: High
```

```
   CWE: CWE-502 (https://cwe.mitre.org/data/definitions/502.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_calls.html#b301-pickle
   Location: .\todo_manager.py:47:15
46          with open(fileName, "rb") as f:
47              return pickle.load(f)
48

--------------------------------------------------
>> Issue: [B605:start_process_with_a_shell] Starting a process with a shell, possible injection detected,
    security issue.
   Severity: High   Confidence: High
   CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b605_start_process_with_a_shell.html
   Location: .\todo_manager.py:57:4
56          # Helper kept for the lab: not called by the demo
57          os.system("mail -s 'Pending tasks' " + address + " < tasks.json")
58

--------------------------------------------------
>> Issue: [B311:blacklist] Standard pseudo-random generators are not suitable for security/cryptographic
    purposes.
   Severity: Low   Confidence: High
   CWE: CWE-330 (https://cwe.mitre.org/data/definitions/330.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_calls.html#b311-random
   Location: .\todo_manager.py:70:17
69          print("Task id for 'gym':", findTask("gym")) # returns None silently
70          suggestion = random.choice(pendingTasks())
71          print("Try this next:", suggestion)

--------------------------------------------------

Run metrics:
        Total issues (by severity):
                Undefined: 0
                Low: 2
                Medium: 1
                High: 1
        Total issues (by confidence):
                Undefined: 0
                Low: 0
                Medium: 0
                High: 4
Files skipped (0):
```

Flake8 (before)

```
todo_manager.py:12:11: E225 missing whitespace around operator
todo_manager.py:40:1: E303 too many blank lines (3)
todo_manager.py:69:49: E261 at least two spaces before inline comment
todo_manager.py:74:38: E231 missing whitespace after ','
```

5. Tool output: after

Pylint (after)

```
--------------------------------------------------------------------
Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)
```

Bandit (after)

```
Run started:2026-10-14 18:08:06.018798+00:00

Test results:
        No issues identified.

Code scanned:
        Total lines of code: 88
        Total lines skipped (#nosec): 0
        Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

Run metrics:
        Total issues (by severity):
                Undefined: 0
                Low: 0
                Medium: 0
                High: 0
        Total issues (by confidence):
                Undefined: 0
                Low: 0
                Medium: 0
                High: 0
Files skipped (0):
```

Flake8 (after)

6. Commit history

```
eb2992f Fix Pylint code quality issues, achieve 10.00/10 score, add reports and documentation
```

```
1241efe Fix Flake8 PEP 8 formatting and style issues
c5d5930 Fix Bandit security issues (B605, B301, B403, B311)
7c281ce Initial commit
```

One commit for each tool, as required by the handout. The full before and after reports are in the `reports/before` and `reports/after` folders of the repository.

7. Reflection

Question 1: What is static code analysis, and how does it differ from dynamic testing in the Software Development Lifecycle (SDLC)?

Reflection & Analysis:

Static code analysis is the process of examining software source code, byte code, or configuration files without executing the program. It relies on parsers, Abstract Syntax Trees (ASTs), Control Flow Graphs (CFGs), and data flow/taint analysis engines to automatically inspect code structure, identify syntax errors, enforce coding standards, detect anti-patterns, and uncover security vulnerabilities early in the Software Development Lifecycle (SDLC).

Comparison with Dynamic Testing:

1. **Execution Requirement:** Static analysis requires only the raw source code and does not need a compiled binary, execution environment, test database, or test inputs. Conversely, dynamic testing (such as unit testing, integration testing, fuzzing, and penetration testing) requires the program to be running in an active execution environment.
2. **Phase of Detection ("Shift-Left"):** Static analysis can be applied instantaneously as code is written in the IDE or during pre-commit checks. This embodies the "shift-left" paradigm, enabling developers to remediate bugs and vulnerabilities when they are fastest and cheapest to fix. Dynamic testing typically occurs later in staging or testing pipelines.
3. **Coverage & Scope:** Static analysis inspects 100% of the code paths regardless of whether they are exercised by test suites. However, static analysis cannot evaluate runtime interactions, memory state, concurrency race conditions, or third-party service latency, which dynamic testing excels at validating.
4. **Conclusion:** Static analysis and dynamic testing are complementary techniques. Static analysis acts as a preventative quality and security gate, while dynamic testing verifies functional behavior and runtime resilience.

Question 2: What are the distinct roles and scopes of Pylint, Flake8, and Bandit, and how do they complement each other?

Reflection & Analysis:

In this lab, three industry-standard static analysis tools were employed, each targeting a distinct facet of code health:

1. **Bandit — The Security Guard (SAST):**
   - *Role:* Dedicated Static Application Security Testing (SAST) tool for Python.
   - *Scope:* It inspects the AST for common security pitfalls, known dangerous functions, insecure configuration defaults, and common weaknesses mapped to the MITRE CWE database (e.g., shell injection, insecure deserialization, weak hashing, hardcoded credentials).
   - *Lab Finding:* Detected `B605` (shell injection in `os.system`), `B301`/`B403` (unsafe `pickle` deserialization), and `B311` (cryptographically insecure PRNG).
2. **Flake8 — The Grammar & Style Checker (PEP 8):**
   - *Role:* Lightweight, fast linter combining `PyFlakes` (logical syntax and import errors), `pycodestyle` (PEP 8 style guide enforcement), and `Ned Batchelder's McCabe script` (cyclomatic complexity).
   - *Scope:* Focuses strictly on code formatting, whitespace around operators, line lengths, blank lines, and unused imports.
   - *Lab Finding:* Flagged missing operator whitespace (`E225`), excessive blank lines (`E303`), insufficient inline comment indentation (`E261`), and missing commas (`E231`).
3. **Pylint — The Strict Code Reviewer (Quality & Architecture):**
   - *Role:* Comprehensive code quality, refactoring, and logical error analyzer.
   - *Scope:* Enforces naming conventions (snake_case vs camelCase), docstring completeness, dangerous defaults (mutable arguments), unreachable code, code complexity, and computes an overall quantitative code score (0 to 10).
   - *Lab Finding:* Flagged mutable default list `labels=[]` (`W0102`), missing docstrings (`C0114`, `C0116`), non-conformant `camelCase` identifiers (`C0103`), implicit return statements (`R1710`), and unencoded file operations (`W1514`), raising the initial rating of 5.09/10 to a clean 10.00/10.

Together, these three tools provide a complete static quality umbrella: **Flake8 ensures consistent aesthetics, Pylint ensures robustness and maintainability, and Bandit ensures vulnerability prevention.**

Question 3: Discuss the high- and medium-severity vulnerabilities and logic bugs identified in `todo_manager.py`. Why are they dangerous in production, and how were they mitigated?

Reflection & Analysis:

1. Command Injection (`os.system` / Bandit B605 / CWE-78 — High Severity)

- **Vulnerability Mechanism:** The `remindByEmail` helper constructed a shell string via concatenation: `os.system("mail -s 'Pending tasks' " + address + " < tasks.json")`. The shell interprets special characters like `;`, `&&`, `|`, and backticks. If an untrusted email address like `attacker@domain.com; rm -rf /` is supplied, the subshell executes the injected command.
- **Production Hazard:** Can lead to full server compromise, data exfiltration, lateral movement, or denial of service.
- **Remediation:** Removed shell execution entirely. Utilized Python's standard `email.message.EmailMessage` object to assemble the email data safely in memory without invoking an external shell.

2. Insecure Deserialization (`pickle.load` / Bandit B301, B403 / CWE-502 — Medium Severity)

- **Vulnerability Mechanism:** Python's `pickle` serialization format is not secure against maliciously crafted data. It is capable of instantiating arbitrary Python callables during unpickling via the `__reduce__` protocol.
- **Production Hazard:** An attacker modifying `tasks.pkl` on disk or sending a malicious payload over a network endpoint can execute arbitrary Python commands inside the host environment.
- **Remediation:** Migrated persistence to structured `JSON` format (`json.dump` / `json.load`). Because JSON represents purely declarative data structures (strings, numbers, dictionaries, lists) without executable bytecode representations, deserialization attacks are fundamentally prevented.

3. Mutable Default Argument (`labels=[]` / Pylint W0102 — High Severity Bug)

- **Vulnerability Mechanism:** Python binds function default arguments once when the function definition is executed, creating a single persistent object instance. In `def addTask(..., labels=[])`, the list was mutated across calls via `labels.append(priority)`.
- **Production Hazard:** Calls that omit `labels` all share one list, so priorities appended by one call would leak into later calls. In this program the list was not stored on tasks, so the bug was latent, but it would surface as soon as labels were used.
- **Remediation:** Set the default value to `None` (`labels=None`). Inside the function body, initialize a new list instance whenever the parameter evaluates to `None` (`if labels is None: labels = []`).

4. Weak Pseudo-Random Number Generation (`random` / Bandit B311 / CWE-330 — Low Severity)

- **Vulnerability Mechanism:** `random.choice` uses the Mersenne Twister algorithm (MT19937), which has a predictable internal state cycle.
- **Production Hazard:** Predictable output would matter if this were used for tokens or keys. Here it only picks a suggested task, so the real risk is low; I switched anyway to clear the finding.
- **Remediation:** Adopted `secrets.choice()` from Python's `secrets` module, which draws from OS-level entropy pools (`os.urandom`) designed specifically for cryptographically secure randomness.

Question 4: How can static analysis tools be effectively integrated into CI/CD pipelines and developer workflows without causing alert fatigue?

Reflection & Analysis:

Automating static analysis within modern DevSecOps pipelines maximizes code reliability while keeping development velocity high:

1. **Local Pre-Commit Hooks:**

- Using frameworks like `pre-commit`, lightweight checks (such as Flake8 formatting and Bandit scans) can be executed automatically on staged files before a commit is created. This catches issues immediately on the developer's workstation before reaching remote repositories.

2. **Automated CI Quality Gates (GitHub Actions / GitLab CI):**
   - CI workflows can run `pylint`, `bandit`, and `flake8` on pull requests.
   - A quality gate can be configured to block merging if:
   - Any High or Medium severity Bandit vulnerabilities are detected.
   - Pylint score drops below an acceptable threshold (e.g., `< 9.5`).
   - Flake8 reports formatting or syntax violations.
3. **Combating Alert Fatigue & False Positives:**
   - *Baseline Configuration:* In legacy codebases, establish a baseline file so new scans only flag newly introduced issues rather than overwhelming teams with hundreds of legacy warnings.
   - *Fine-Tuning Rule Sets:* Use configuration files (`.pylintrc`, `.flake8`, `bandit.yaml`) to disable rules that do not align with team architectural decisions (e.g., adjusting maximum line lengths from 79 to 100, or ignoring certain non-security warnings in test directories).
   - *Inline Suppression with Justification:* In genuine false-positive scenarios, use explicit comments with justification (e.g., `# nosec BXXX: sanitized by regex above`) rather than disabling rules globally.